

实验 5：存储器及数据通路设计实验报告

姓名： 学号：

2026 年 5 月 12 日

1 实验目的

1. 掌握利用 ROM 组件实现指令存储器的设计方法。
2. 掌握 ASCII 点阵字库的使用方法。
3. 掌握利用 RAM 组件实现按字节存取的数据存储器的设计方法。
4. 掌握实现 RV32I 存取指令的设计方法。
5. 掌握 RV32I 下地址逻辑部件的设计方法。
6. 掌握 RV32I 指令译码、反汇编的实现方法。
7. 掌握 RV32I 立即数扩展器的设计方式。
8. 根据 RV32I 指令执行过程，掌握 RV32I 运算类指令数据通路的设计方法。
9. 掌握跳转控制器的设计方法。

2 实验环境

- 软件环境：Logisim
- 实验平台：数字逻辑与计算机基础实验平台

3 实验内容与结果分析

3.1 Lab5.1：只读存储器实验

3.1.1 实验原理

本实验利用 ASCII 码可显示字符点阵字库（ascii8-16.zk），通过 ROM 组件在 Logisim 的 LED 点阵组件上显示 ASCII 码字符形状。

点阵字库从字符空格 SP (ASCII 码值 0x20) 开始, 到删除字符 DEL (ASCII 码值 0x7f) 共 96 个字符, 每个字符使用 8 列 16 行的点阵表示, 共占 16 个字节。因此字库总大小为 $96 \times 16 = 1536$ 字节。

根据输入的 ASCII 码查找字库中对应起始地址的方法为: 将 ASCII 码值减去 0x20, 再将差值左移 4 位 (即乘以 16), 即得该字符点阵在字库中的起始地址。

为了同时输出某个字符 16 个字节的点阵数据, 需要复制 16 个加载相同点阵字库的 ROM, 各 ROM 的地址依次加 1, 从而同时读取第 1 行到第 16 行的点阵数据并同步输出到 LED 点阵组件。

ROM 数据位宽设置为 8 位, 地址位宽至少为 11 位; 片选信号设置为高电平有效。若输入 ASCII 码不在可见字符范围内, 则输出错误标志位 CodeErr。

3.1.2 电路设计

在工作区中按照实验布局图放置 ASCII-CODE 输入引脚、16 个 ROM 组件、加减法器、CodeErr 错误标志输出引脚及 LED 点阵组件, 并完成连线。16 个 ROM 均加载 ascii8-16.zk 点阵字库文件, 各 ROM 地址端口分别接字符基地址依次加 0 到 15 的偏移值。

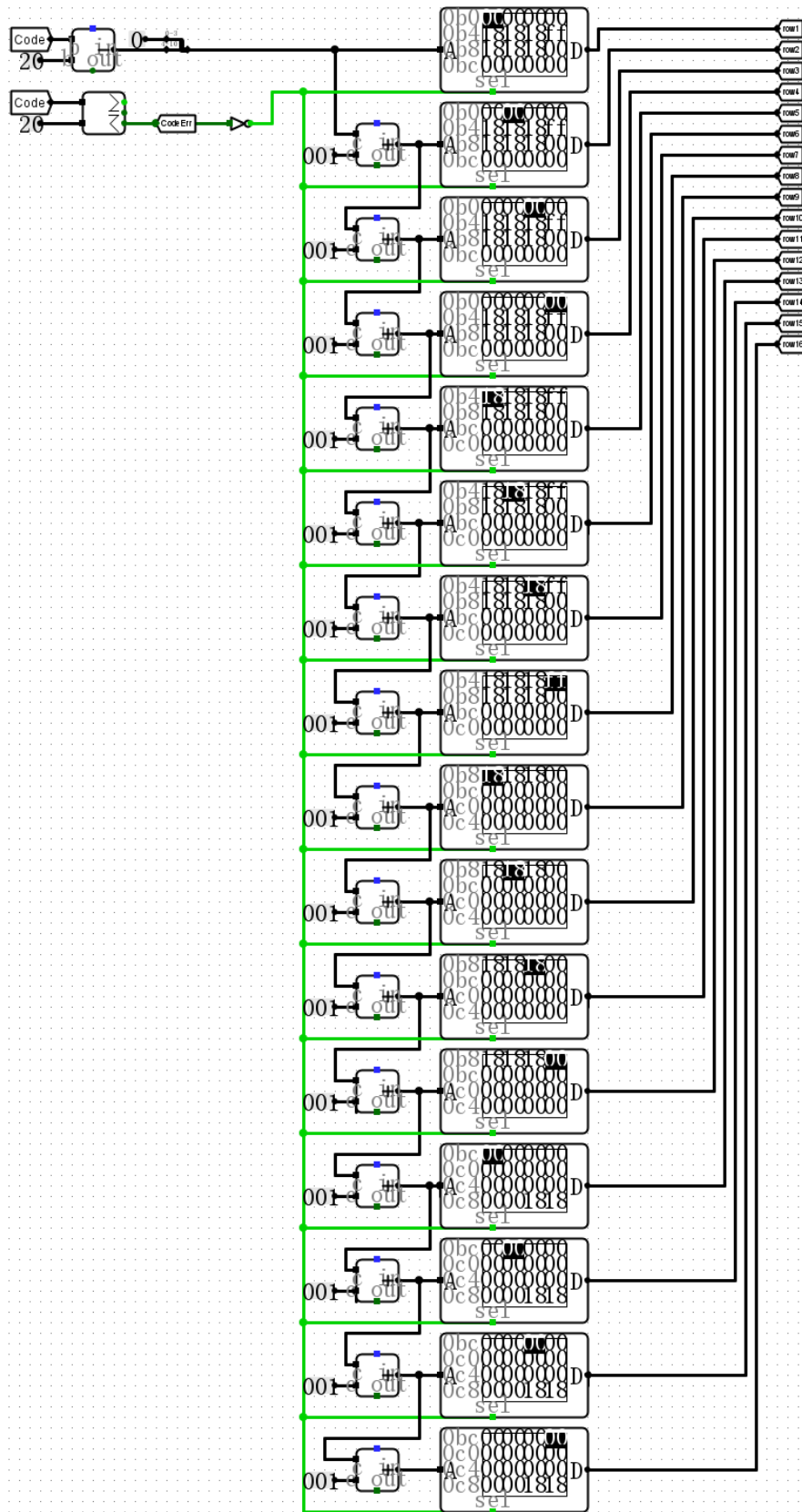


图 1: 只读存储器实验电路图

3.1.3 仿真测试与分析

输入不同的 ASCII 码值，观察 LED 点阵上显示的字符形状。对可见字符范围内的输入，LED 点阵正确显示对应字符；对不在范围内的输入，CodeErr 标志位有效，LED 点阵无有效显示。仿真结果与理论预期一致。

实验 5：存储器及数据通路设计实验报告

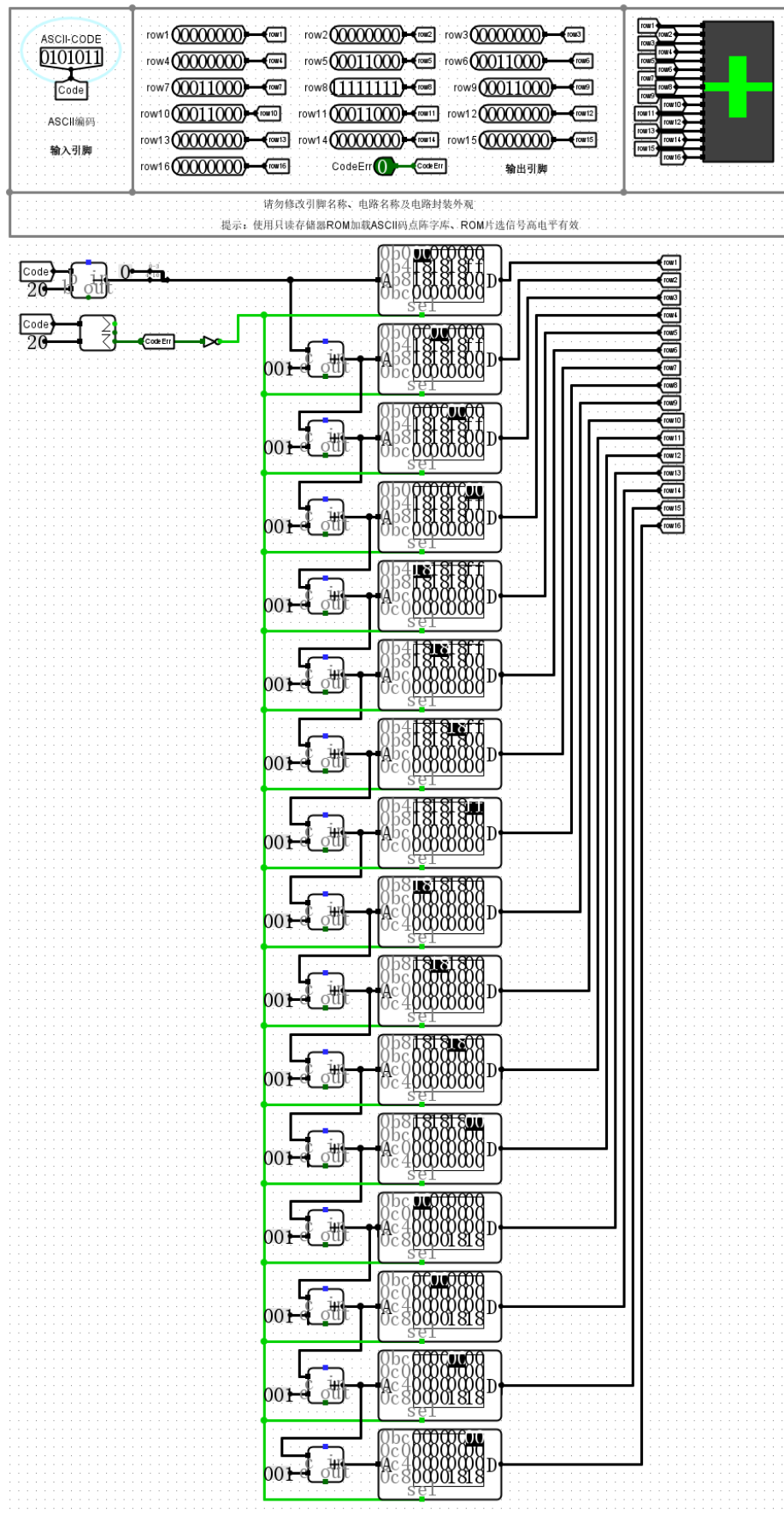


图 2：只读存储器实验仿真测试图

3.2 Lab5.2: 数据存储实验

3.2.1 实验原理

RV32I 指令集要求数据存储支持按字节 (byte)、半字 (halfword) 和字 (word) 进行读写, 因此引入控制信号 MemOp 来表示当前访存操作的字节长度, 其编码定义如表 1 所示。

表 1: MemOp 控制信号含义

MemOp	指令	含义
000	lw, sw	按字存取, 4 字节
001	lbu	按字节读取, 1 字节, 0 扩展到 4 字节
010	lhu	按半字读取, 2 字节, 0 扩展到 4 字节
101	lb, sb	按字节存取, 1 字节, 读取时符号位扩展到 4 字节
110	lh, sh	按半字存取, 2 字节, 读取时符号位扩展到 4 字节

实验设计 256KB 数据存储, 数据字长 32 位, 地址位宽 18 位。采用 4 片数据位宽为 8 位的 RAM (RAM3~RAM0) 级联实现 32 位存储。根据 MemOp 和地址最低 2 位 Addr[1:0] 确定片选信号 SEL3~SEL0 及地址对齐错误标志 AlignErr, 具体对应关系见实验指导表 5.2。

3.2.2 辅助电路设计 (ROM 实现片选逻辑)

在本实验中, 需要根据 MemOp[2:0] 和 Addr[1:0] (共 5 位输入) 来生成片选信号 SEL3~SEL0 和 AlignErr (共 5 位输出)。将这一组合逻辑直接用逻辑门实现较为繁琐, 因此本实验巧妙地使用了一个 ROM 来代替复杂的逻辑门网络。

具体方法是: 将 MemOp[2:0] 和 Addr[1:0] 拼接成 5 位地址输入到 ROM, ROM 的数据位宽设置为 5 位, 将所有 32 种输入组合对应的 {SEL3, SEL2, SEL1, SEL0, AlignErr} 输出值预先写入 ROM 中。这样, ROM 即充当一张完整的真值表, 只要输入地址, 就能直接查表输出正确的控制信号, 电路结构简洁, 且易于验证与修改。辅助电路如图 3 所示。

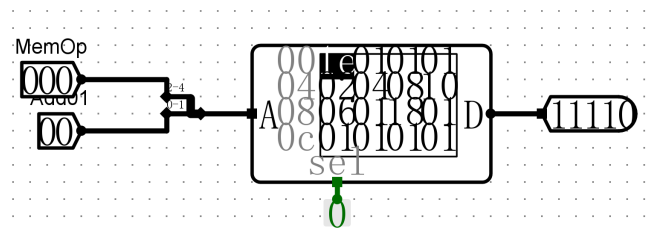


图 3: 利用 ROM 实现 MemOp 与 Addr 到片选信号转换的辅助电路

3.2.3 电路设计

在 Logisim 主电路工作区中放置 4 个 RAM 组件（数据位宽 8 位，地址宽度 16 位，Separate load and store ports 模式，高电平片选），以及多路选择器、位扩展器、逻辑门、隧道等组件，并接入辅助电路的片选信号输出，完成整体电路设计。

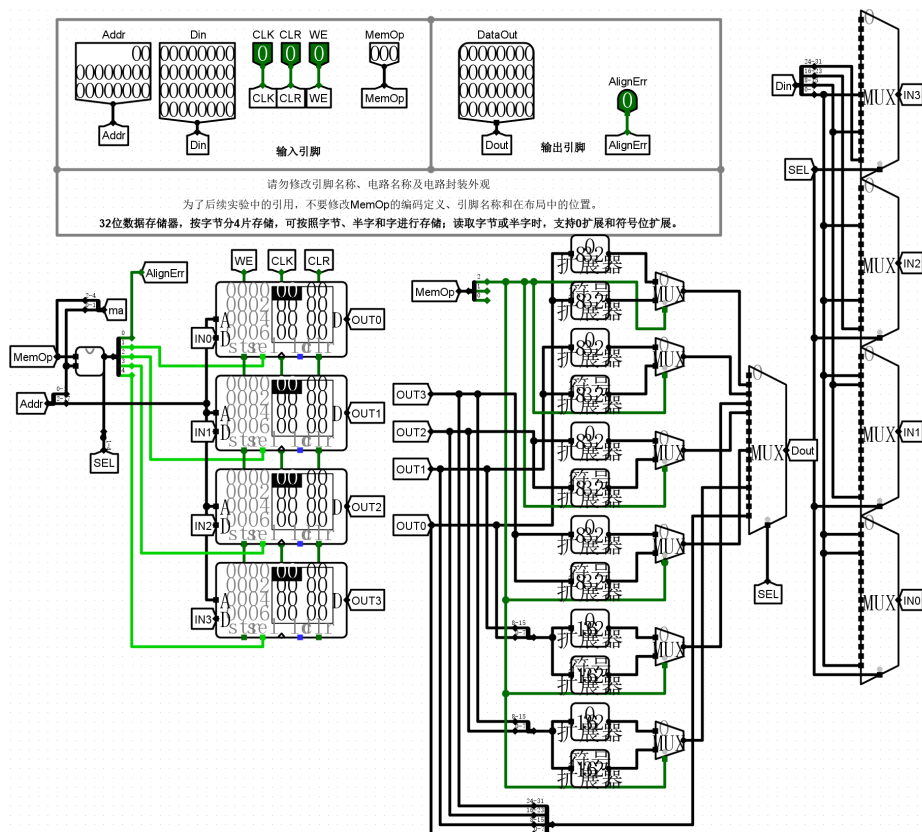


图 4：数据存储器实验电路图

3.2.4 仿真测试与分析

设置不同的 MemOp、地址和输入数据，分别测试字存取、半字存取（含符号扩展与零扩展）、字节存取（含符号扩展与零扩展）等功能，验证地址对齐异常时 AlignErr 有效。仿真结果与理论预期一致。

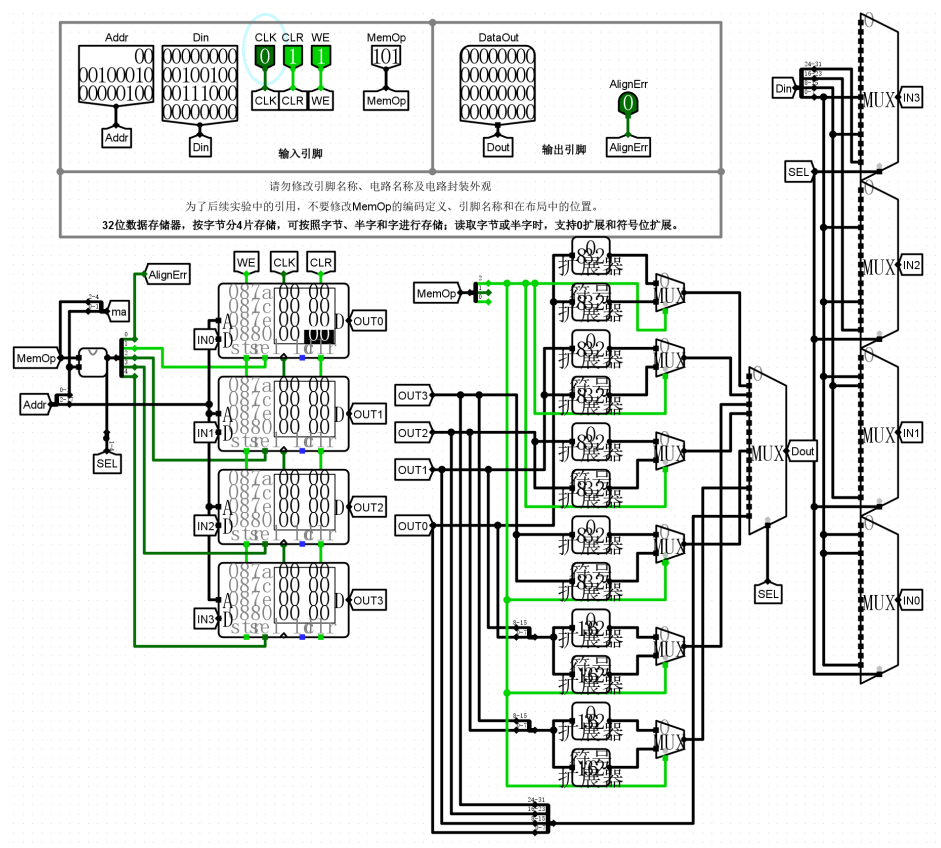


图 5: 数据存储器实验仿真测试图

3.3 Lab5.3: 取指令部件 IFU 实验

3.3.1 实验原理

取指令部件 (Instruction Fetch Unit, IFU) 负责根据程序计数器 PC 的值从指令存储器中读取当前指令, 并计算下一条指令地址。

下一条指令地址的计算受控制信号 NPCASrc 和 NPCBSrc 控制:

- NPCASrc = 0 时, 加法器 A 端选择 PC; NPCASrc = 1 时, 选择 BusA (rs1 寄存器值)。
- NPCBSrc = 0 时, 加法器 B 端选择常量 4; NPCBSrc = 1 时, 选择立即数 Imm。

由于 Logisim ROM 按字长编址, 而 RV32I 指令地址按字节编址, 因此将 PC[17:2] 送至 ROM 地址端口即可 (丢弃最低 2 位)。

初始地址 Initial Address 在复位信号 Reset 高电平有效后, 于下一个时钟上升沿加载到 PC 寄存器中。

3.3.2 电路设计

在工作区中放置 PC 寄存器（32 位，上升沿触发，使能信号始终有效）、指令存储器 ROM（数据位宽 32 位，地址位宽 16 位，高电平片选）、加法器、多路选择器及隧道等组件，按原理完成连接。

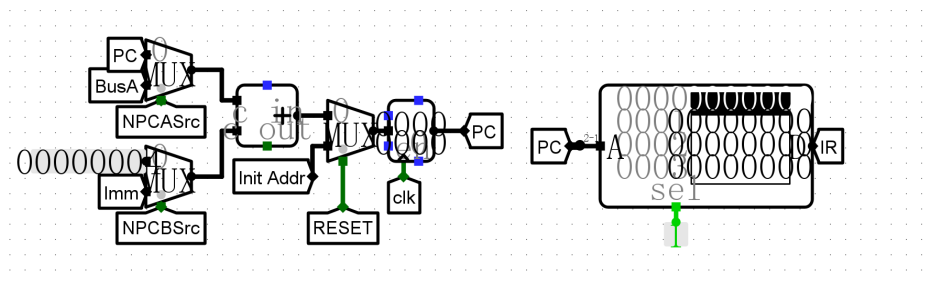


图 6: 取指令部件 IFU 电路图

3.3.3 仿真测试与分析

加载指令存储器镜像文件 lab5.3.hex，按照表 5.4 依次设置输入信号，单步执行，观察 PC 和 IR 输出，验证取指令部件在复位、顺序执行、立即数跳转及寄存器跳转等情况下均能正确工作。

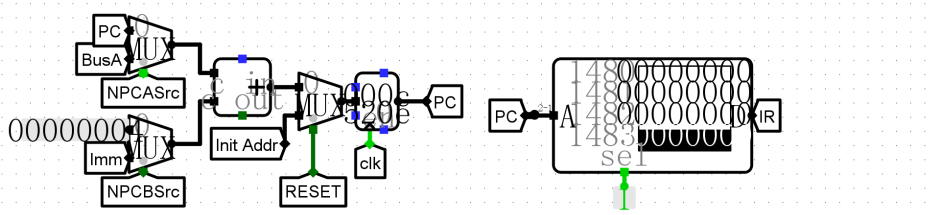
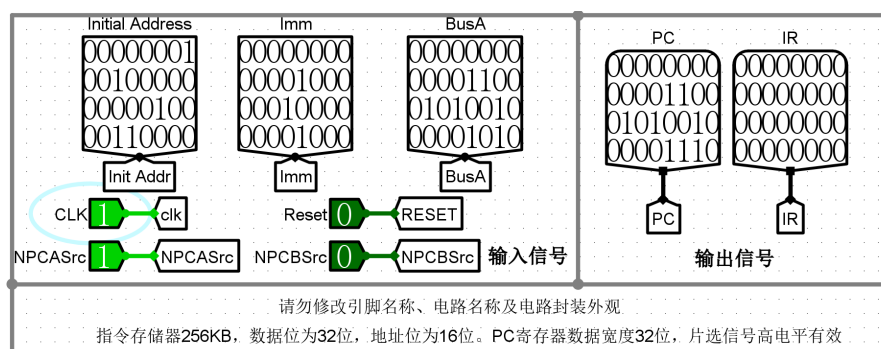


图 7: 取指令部件 IFU 仿真测试图

3.4 Lab5.4: 取操作数部件 IDU 实验

3.4.1 实验原理

取操作数部件 (Instruction Decode Unit, IDU) 主要完成指令译码、立即数扩展、寄存器堆读取及 ALU 操作数选择等功能。

RV32I 中除 R 型指令外, 其余 5 种指令格式均含立即数, 立即数扩展器根据 ExtOp 控制信号选择不同类型的扩展方式:

- ExtOp = 0 (I-型): $\text{immI} = \{20\{\text{Instr}[31]\}, \text{Instr}[31:20]\}$
- ExtOp = 1 (U-型): $\text{immU} = \{\text{Instr}[31:12], 12'b0\}$
- ExtOp = 2 (S-型): $\text{immS} = \{20\{\text{Instr}[31]\}, \text{Instr}[31:25], \text{Instr}[11:7]\}$
- ExtOp = 3 (B-型): $\text{immB} = \{19\{\text{Instr}[31]\}, \text{Instr}[31], \text{Instr}[7], \text{Instr}[30:25], \text{Instr}[11:8], 1'b0\}$
- ExtOp = 4 (J-型): $\text{immJ} = \{11\{\text{Instr}[31]\}, \text{Instr}[31], \text{Instr}[19:12], \text{Instr}[20], \text{Instr}[30:21], 1'b0\}$

ALUASrc 控制 ALU A 端输入选择 (0: BusA; 1: PC), ALUBSrc 控制 B 端输入选择 (00: BusB; 01: 常数 4; 10: 立即数 Imm)。

3.4.2 立即数扩展器电路设计

在 Logisim 中构建”立即数扩展器”子电路, 输入为 32 位指令字 Instr 和 3 位控制信号 ExtOp, 输出为 32 位立即数 imm。使用分线器拆分指令各字段, 通过拼接器和多路选择器实现五种扩展方式。

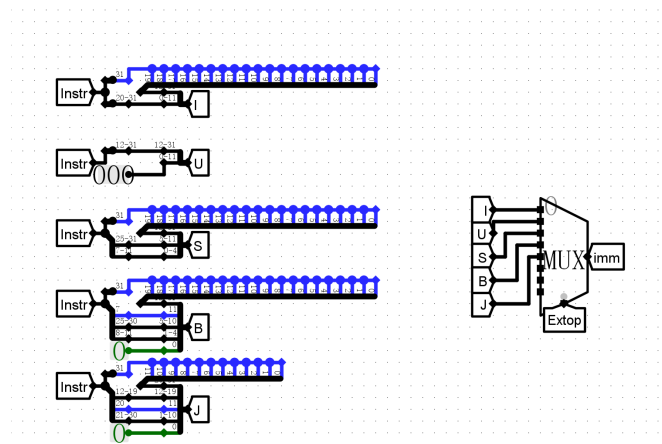


图 8: 立即数扩展器电路图

3.4.3 IDU 整体电路设计

在主电路工作区中放置立即数扩展器子电路、寄存器堆 RegFile 子电路（0 号寄存器硬连为零，上升沿触发）、多路选择器、分线器及隧道等组件，完成 IDU 整体电路的连接。

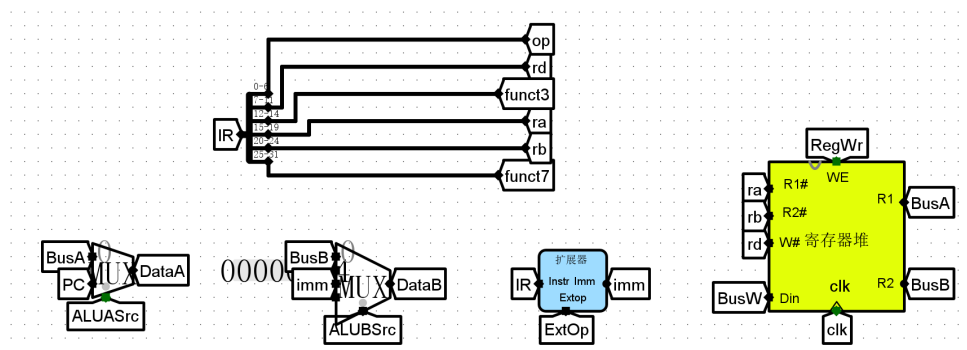


图 9: 取操作数部件 IDU 电路图

3.4.4 仿真测试与分析

依次输入表 5.5 中的指令码和控制信号，单步执行后，通过 RV32I 指令集手册对各指令进行反汇编，验证 DataA、DataB 等输出值，以及寄存器堆写回结果与理论分析一致。

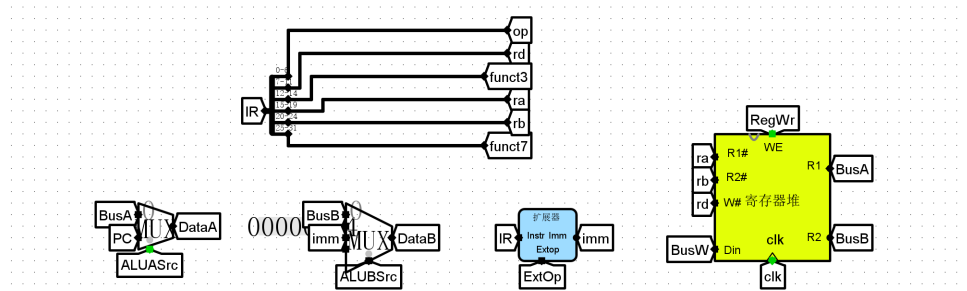
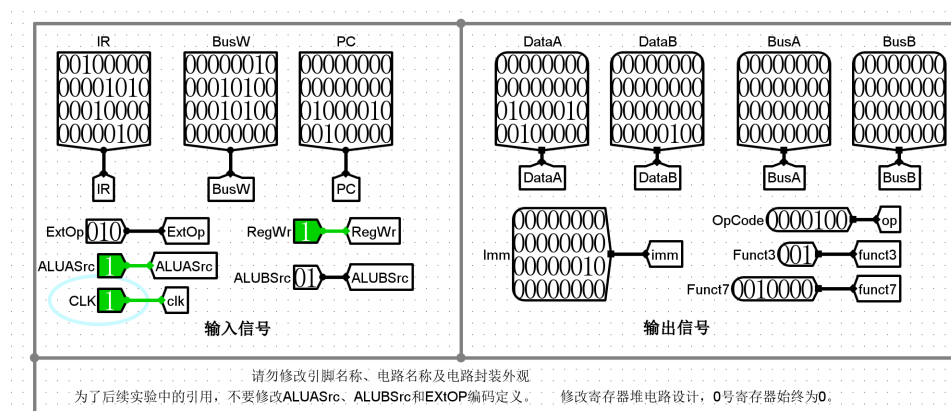


图 10: 取操作数部件 IDU 仿真测试图

3.5 Lab5.5：数据通路实验

3.5.1 实验原理

单周期 CPU 数据通路是将取指令部件 IFU、取操作数部件 IDU、ALU、数据存储器 DataRAM 以及跳转控制器 BandJ 等部件连接起来，共同完成 RV32I 指令的执行过程。

跳转控制器 BandJ 根据控制信号 BandJ 编码和 ALU 输出的 Zero 及 Result[0] 信号，决定 NPCASrc 和 NPCBSrc，其编码定义如表 2 所示。

表 2: BandJ 控制信号含义

指令类型	BandJ	NPCASrc	NPCBSrc
非跳转指令	000	0	0
jal (无条件跳转 PC 目标)	001	0	1
jalr (无条件跳转寄存器目标)	010	1	1
beq (等于分支)	100	0	Zero
bne (不等于分支)	101	0	!Zero
blt/bltu (小于分支)	110	0	Result[0]
bge/bgeu (大于等于分支)	111	0	Zero (!Result[0])

3.5.2 BandJ 跳转控制器电路设计

在 Logisim 中构建“BandJ”子电路，输入为 3 位 BandJ 控制信号、1 位 Zero 和 1 位 Result0，输出为 NPCASrc 和 NPCBSrc。根据表 2 推导逻辑表达式，通过分线器和逻辑门完成电路连接。

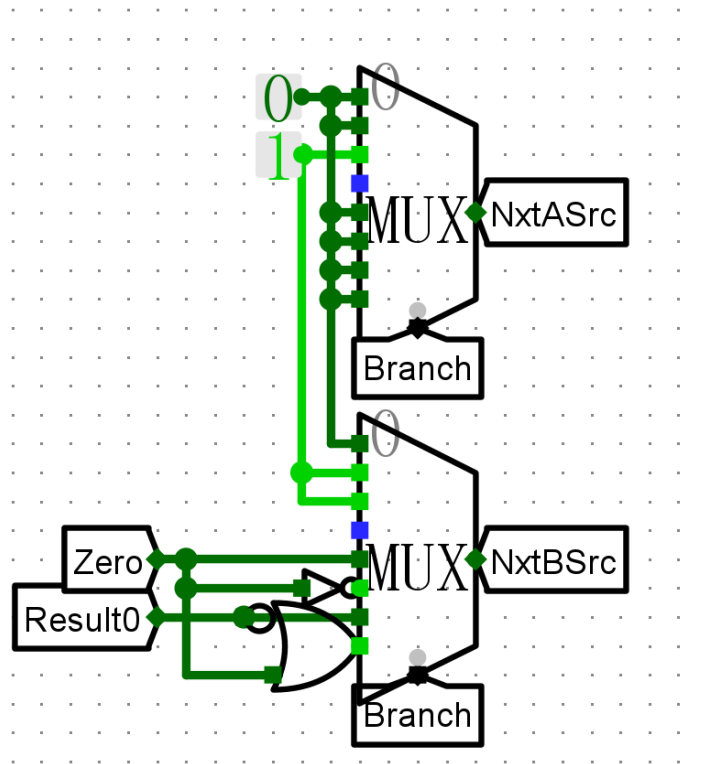


图 11: BandJ 跳转控制器电路图

3.5.3 BandJ 仿真测试

改变 BandJ、Zero、Result0 输入，验证 NPCASrc 和 NPCBSrc 输出与表 2 定义一致。

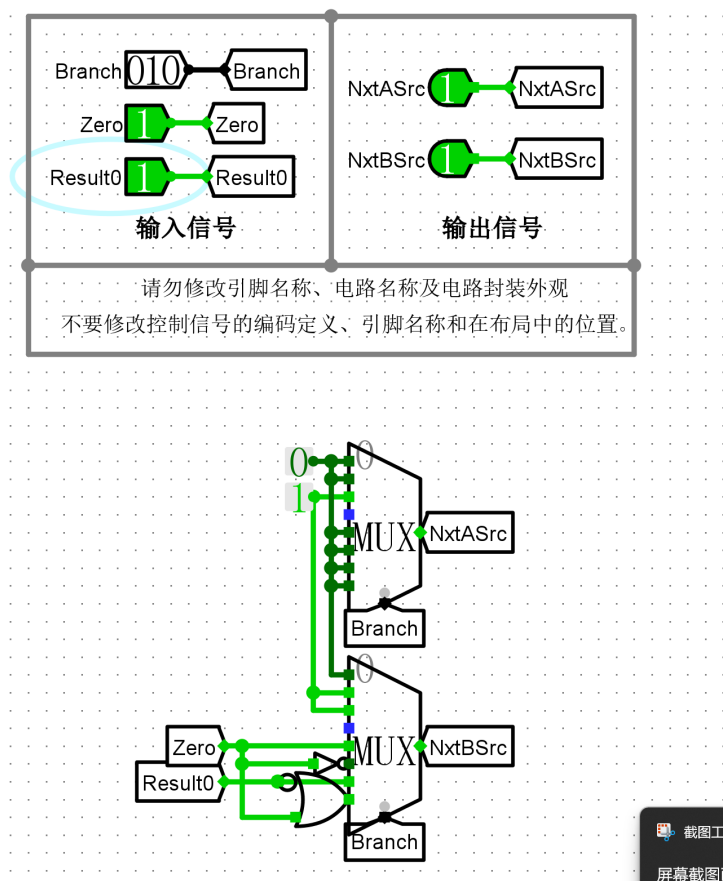


图 12: BandJ 跳转控制器仿真测试图

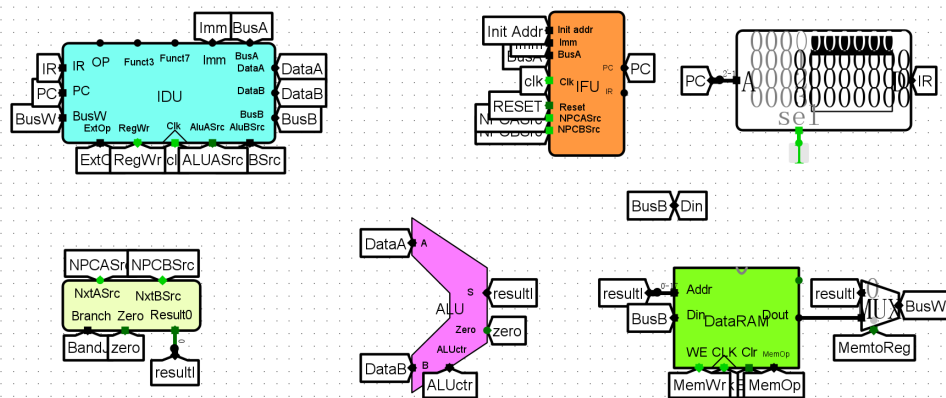
3.5.4 数据通路整体电路设计

在 Logisim 主电路工作区中，加载 lab4.5、lab5.2、lab5.4 等库文件，放置 IFU 子电路、指令存储器 ROM（256KB，32 位字长，16 位地址，高电平片选，片选始终有效）、BandJ 子电路、IDU 子电路、ALU 子电路和数据存储器 DataRAM 子电路，并添加输入输出引脚、隧道和集线器等组件。

PC 连接到指令存储器地址端时取 PC[17:2]；ALU 输出连接到数据存储器地址端时取低 18 位；数据存储器清零信号连接到复位信号 ReSet。根据图 5.7 所示单周期 CPU 数据通路原理图完成所有组件的连接。



在指令存储器第 0x100 单元起写入 21 条 RV32I 指令机器码（或加载镜像文件 lab5.5.hex），设置初始地址为 0x400，复位信号有效后取消，单步执行依次读取并执行各指令，根据表 5.7 提供的控制信号赋值，记录 PC、BusW、Din、NPCASrc、NPCBSrc 等输出，观察寄存器堆和数据存储器的变化。实验结果表明，数据通路可以正确执行包括算术运算、逻辑运算、访存操作及跳转指令在内的多种 RV32I 指令。



15

3.5.6 错误分析

在数据通路测试过程中，一开始出现了指令执行结果异常的问题。经排查，发现根本原因在于指令存储器 ROM 中加载的数据镜像文件不正确——加载的文件偏移与实验要求的起始地址 0x100 不匹配，导致 PC 取到的指令码错误，进而引发后续所有执行结果与预期不符。

分析清楚原因后，重新整理了 lab5.5.hex 文件的格式，确保前 256 个字（地址 0x000~0x0FF）填充为 0，从第 0x100 单元起依次写入 21 条指令机器码，再次加载到 ROM 中。重新仿真后，所有指令均能正确执行，测试数据与理论分析完全吻合，通过了全部测试。

4 实验总结

通过本次实验，我完成了只读存储器（ROM）点阵显示、数据存储器（RAM，按字节/半字/字存取）、取指令部件（IFU）、取操作数部件（IDU）以及单周期 CPU 数据通路的设计与验证。实验过程中，我进一步掌握了以下内容：

1. 理解 Logisim 中 RAM 和 ROM 的使用方式，包括十六进制编辑器与镜像文件加载；
2. 掌握 ASCII 点阵字库的组织方式及基于 ROM 的字符显示原理；
3. 学会利用 ROM 实现复杂组合逻辑的查表映射，简化数据存储器片选控制电路；
4. 理解 RV32I 指令格式，掌握指令译码、立即数扩展和寄存器堆读写的实现；
5. 掌握程序计数器、下地址逻辑与跳转控制器的设计；
6. 通过将各子部件集成，完整构建了支持 RV32I 整数运算指令的单周期 CPU 数据通路。

本次实验将存储器、运算部件与控制逻辑综合起来，是迄今最复杂的综合性实验，对深入理解计算机组成原理和处理器数据通路设计有重要意义。

5 思考题

5.1 1. 如何利用 ROM 实现滚动显示的功能，在 3 个 LED 点阵矩阵中左右滚动显示 5 个 ASCII 字符

以“NJUCS”为例，5 个字符共 $5 \times 8 = 40$ 列点阵数据。若要在 3 个 LED 点阵（共 24 列）中实现左右滚动显示，可采用如下方案：

将 5 个字符的全部点阵数据依次写入一个较大的 ROM 中, 并引入一个计数器作为”窗口起始列”, 每次时钟跳变时计数器加 1 (或减 1), 形成循环。通过 3 组 ROM (或地址偏移) 分别读取窗口内连续 8 列的点阵数据, 输出到 3 个 LED 点阵组件, 即可实现滚动效果。当计数器到达最大偏移时回绕, 实现无缝循环滚动。

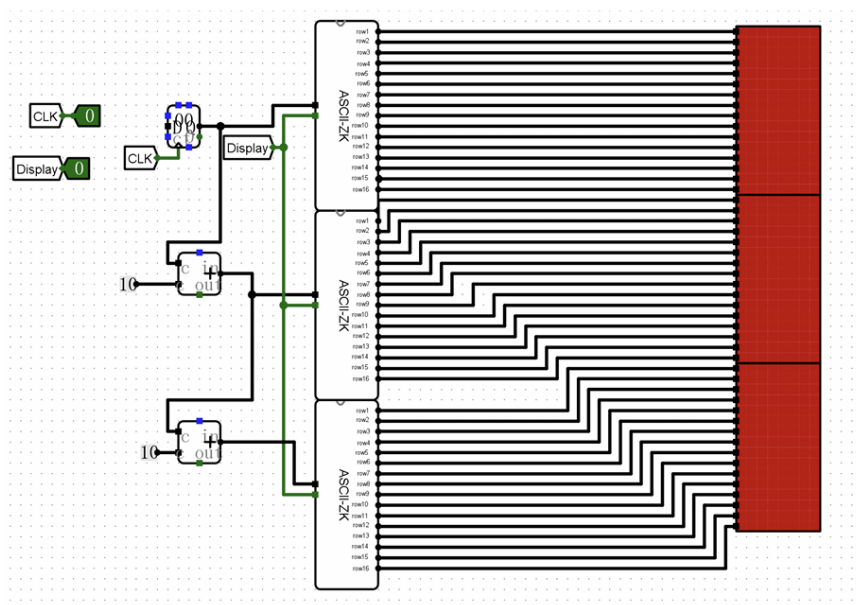


图 15: 思考题 1: 滚动显示方案电路示意图

5.2 2. 分析说明如果 PC 寄存器、寄存器堆和数据存储器写入数据的时钟信号触发边沿不一致, 对程序执行结果有什么影响

在单周期 CPU 中, 所有的写操作 (PC 更新、寄存器堆写回、数据存储器写入) 都应在同一时钟边沿完成, 以保证每个周期内读操作使用的是”当前周期”的数据, 写操作在周期末统一生效。

若触发边沿不一致, 例如 PC 在上升沿更新而寄存器堆在下降沿写回, 则可能出现以下问题:

- 寄存器堆写回发生在 PC 更新之后 (同一周期内), 导致下一条指令取指时寄存器堆已包含本周期写回的数据, 破坏了”本周期读旧值、周期末写新值”的单周期假设, 造成数据竞争。
- 数据存储器若与 PC 触发边沿不一致, 存储指令 (sw/sh/sb) 写入的时机与取指周期不对齐, 可能导致后续加载指令 (lw/lb 等) 读到错误的旧数据或意外的新数据。
- 总体上, 各部件触发边沿不一致会破坏单周期处理器”一个周期内完成一条完整指令”的时序约束, 引发功能错误, 使程序执行结果不可预期。

因此，在设计中应统一所有存储单元的时钟触发边沿（本实验中统一设置为上升沿触发），以保证电路功能正确。

5.3 3. 在 CPU 启动执行后，如何实现在当前程序结束后，CPU 不再继续往下执行

可以采用以下几种方案：

- (1) **无限循环跳转指令**：在程序末尾插入一条 `jal x0, 0`（即跳转到自身地址的无条件跳转指令），使 PC 永远停在该地址，CPU 不断重复执行该空循环指令，不会继续向下访问无效指令。
- (2) **停机标志信号**：增加一个特殊的 HALT 指令，当控制器检测到该指令时，输出一个停机信号，通过该信号屏蔽 PC 的使能端（将 PC 写使能置 0），使 PC 不再更新，CPU 停止取指。
- (3) **判断指令地址越界**：在 IFU 中加入地址范围判断逻辑，当 PC 超出程序存储范围后，将 PC 锁定在最后一条有效地址，或输出停机信号，阻止继续执行。

在 Logisim 实验环境中，最简单可行的方法是方案 (1)，即在程序末尾写入 `jal x0, 0`（机器码 `0x0000006f`），使程序在结束后原地循环，不影响寄存器堆和数据存储器的最终状态。